DATA STRUCTURES & ALGORITHMS

Longest Increasing Subsequence (Printing) using Patience Sorting + Parent Tracking

INTUITION

- Instead of storing the actual LIS, store the index of the smallest possible ending element for every LIS length.
- Use binary search to find where the current element belongs.
- Maintain a parent array to remember the previous element in the LIS.
- After processing all elements, reconstruct the LIS by following the parent pointers backwards.

DATA STRUCTURES USED

1. tailIndex

`vector<int> tailIndex;`

Stores:

`tailIndex[len]` = Index of the smallest ending element of an increasing subsequence of length `(len + 1)`.

It stores indices, not values.

2. parent

`vector<int> parent(n, -1);`

Stores:

`parent[i]` = Previous index in the LIS ending at `i`.

Used only for reconstruction.

ALGORITHM

Step 1 — Initialize

```
tailIndex = {}
parent = {-1,-1,...}
```

Step 2 — Traverse Every Element

```
for(i=0;i<n;i++)
```

Step 3 — Binary Search on tailIndex

Use binary search on `tailIndex`. Search for the first element whose value `>= nums[i]`

Since `tailIndex` stores indices, compare `nums[index]` instead of `index` using the custom comparator.

Step 4 — If No Such Element Exists

`it == tailIndex.end()`

append the current index.

```
tailIndex.push_back(i);
```

This means: A longer LIS has been found.

Step 5 — Otherwise (Replace)

Replace

```
*it = i;
```

Meaning: For this LIS length, a smaller ending element has been found.
A smaller ending element gives more chances to extend the LIS later.

Step 6 — Set Parent Pointer

If the current element is not the first element of an LIS (`pos > 0`), then:

```
parent[i] = tailIndex[pos-1];
```

Meaning: The previous element of the current LIS = Last element of the best LIS having one smaller length.

Step 7 — Identify LIS Tail

After processing all elements, the last element of the LIS is:

`tailIndex.back()`

Step 8 — Reconstruct the LIS

```
cur = tailIndex.back();
while(cur!=-1){
    ans.push_back(nums[cur]);
    cur = parent[cur];
}
```

Step 9 — Reverse the Answer

```
reverse(ans.begin(),ans.end());
```

because reconstruction starts from the last element.

DEEP-DIVE EXPLANATIONS

Why Binary Search Works

`tailIndex` always represents increasing tail values.
Although it stores indices, their corresponding values:

```
nums[tailIndex[0]] < nums[tailIndex[1]] < ... < nums[tailIndex[k]]
```

are always sorted. Therefore, Binary Search can be applied.

Why Replace Instead of Append?

Suppose sequence tails are `2 5` and current element is `3`.
Replace:

```
2 5  ↓  2 3
```

Length remains `2`, but `3 < 5`, so future elements have a better chance to extend the subsequence.

Why Store Indices Instead of Values?

Values cannot tell:

- where they came from.
- which element was previous.

Indices allow `parent[i]` to connect elements together. Without indices, printing the LIS is impossible.

Reconstruction Step Summary

Start from `tailIndex.back()`
Follow `parent[cur]` until `-1`
Collect elements.
Reverse the collected sequence.

COMPLEXITY ANALYSIS

Metric	Complexity	Breakdown
Time Complexity	**O(n log n)**	Binary Search for every element: `O(log n)` across `n` total elements.
Space Complexity	**O(n)**	`tailIndex`: `O(n)`, `parent`: `O(n)`, `answer`: `O(n)`. Overall: `O(n)`

INTERVIEW KEY POINTS (MUST REMEMBER)

1. `tailIndex` stores indices, not values.
2. `tailIndex[len]` = index of the smallest ending element for an LIS of length `len + 1`.
3. Binary search is performed on the values `nums[tailIndex[i]]`, not on the indices themselves.
4. If no larger/equal tail exists, append the current index (LIS length increases).
5. Otherwise, replace the existing tail index with the current index (same LIS length, better/smaller ending value).
6. `parent[i]` stores the previous index in the LIS ending at `i`.
7. When placing an element at position `pos`, set `parent[i] = tailIndex[pos - 1]` (if `pos > 0`).
8. The last element of the final LIS is `tailIndex.back()`.
9. Follow the parent pointers backward to reconstruct the sequence, then reverse it.
10. Time Complexity: `O(n log n)`, Space Complexity: `O(n)`.

These 10 points summarize the entire algorithm and are usually sufficient to explain the approach in an interview before writing the code.